# AGENTS.md

Guía para agentes de IA que trabajen en el proyecto **Escape Room 7°B 2026** del IPEATyM 186 Castagnari.

# Fuente de verdad

1. `docs/escape-room.md` — requisitos funcionales y modelo de datos. **Tiene prioridad ante cualquier otro documento o prototipo.**
2. `docs/ESCAPE ROOM.pdf` — preguntas para cargar como contenido y pistas para leer en papel.
3. `docs/src/index.html` — boceto visual y de flujo de las estaciones.
4. `docs/src/ranking-puntajes/ranking-puntajes.html` — boceto visual de la pantalla final y del ranking.
5. `docs/ideas-escape-room.md` — ideas para el stand (redes sociales, banner, ping pong). **No son requisitos de la aplicación.**

Si este archivo contradice a `docs/escape-room.md`, ver el apartado de este archivo de decisiones registradas, y preguntame para darte el ok y hay que actualizar este archivo resolviendo las dudas dejandolas en esa misma sección de decisiones registradas.

> Nota: `AGENTS.md` mencionaba antes un archivo `docs/REGLAS ESCAPE ROOM.pdf` que **no existe** en el repositorio. Si aparece más adelante, incorporarlo como fuente complementaria, nunca por encima de `docs/escape-room.md`.

# Memoria

Recuerda actualizar siempre el archivo MEMORY.md con el estado actual del proyecto.

# Objetivo del producto

Aplicación web para una feria escolar. Equipos de hasta 4 integrantes recorren 5 estaciones en orden, responden 4 preguntas por estación y compiten por completar el recorrido en el menor tiempo posible.

Temáticas de las estaciones:

1. Redes
2. Hardware
3. Internet
4. Ciberseguridad
5. Sistemas Operativos

Reglas principales:

- Una página de bienvenida pide **nombre del equipo**, **cantidad de integrantes (1–4)** y **dificultad** (`inicial`, `intermedio`, `avanzado`).
- Cada estación muestra **4 preguntas** del nivel elegido; deben estar todas correctas para avanzar.
- La selección de esas 4 preguntas sigue la regla de "Selección de preguntas por estación".
- Cada estación dura **4 minutos**. Al agotarse el tiempo, el equipo queda eliminado.
- **Dos respuestas incorrectas** en una estación eliminan al equipo. Ver "Decisiones registradas".
- Abandonar el juego también elimina al equipo.
- Se muestra siempre un **tiempo general ascendente** y un **tiempo de estación descendente**.
- En cada página de estación se muestra: **cantidad de ayudas**, **cantidad de respuestas incorrectas** y **cantidad de envíos del formulario**. Al llegar a 2 envíos solo queda disponible el botón de finalizar juego por respuestas incorrectas.
- Las ayudas se piden con los botones **Ayuda 1** y **Ayuda 2** (con confirmación). Al usar las 2 se deshabilitan. La app **no muestra el texto de la pista**: los estudiantes de 7.º la leen en papel.
- A los **30 segundos restantes** de la estación: reproducir un audio de aviso y hacer titilar un texto debajo del contador.
- Al completar las 5 estaciones se calcula el puntaje y se muestra el ranking.
- Un equipo puede realizar varios intentos; el puntaje está asociado a cada intento.
- En el aula habrá una notebook conectada a un televisor mostrando el ranking de forma permanente.

## Selección de preguntas por estación

Siempre se muestran **4 preguntas**, todas del nivel del intento. El algoritmo es:

1. Tomar los temas activos de la estación.
2. Si hay **más de 4 temas**, elegir 4 temas al azar y una pregunta de cada uno.
3. Si hay **4 temas o menos**, elegir al menos una pregunta de cada tema y completar los lugares restantes al azar con preguntas de la estación y nivel.
4. Persistir la selección y el orden en `intento_pregunta`.

Reglas de contenido:

- **Máximo 4 temas por estación**, validado en `/admin`.
- Cada tema activo debe tener **al menos 1 pregunta del nivel**.
- Cada estación debe tener **al menos 4 preguntas del nivel** en total.
- `inicial` siempre está habilitada y debe tener cobertura completa.
- `intermedio` y `avanzado` solo se habilitan si **todas** las estaciones tienen cobertura de ese nivel.
- El contenido actual (`docs/ESCAPE ROOM.pdf`) tiene 4 preguntas por estación y un solo nivel, así que solo `inicial` puede habilitarse hasta que se carguen las otras dificultades.
- La selección se congela por intento: recargar la página no cambia las preguntas.

# Stack y arquitectura

- **Backend:** Python + Flask.
- **Base de datos:** MySQL.
- **Frontend:** HTML server-rendered con Jinja, Bootstrap.
- **Estilos:** Bootstrap mínimo, solo lo necesario.
- **Sin SPA ni frameworks frontend** (React, Vue, etc.). No agregar dependencias pesadas sin consultar.

El servidor es la autoridad del juego:

- Validar respuestas y orden de estaciones en el backend.
- No enviar respuestas correctas al navegador ni confiar en validaciones JavaScript.
- Calcular tiempos y ranking a partir de fechas guardadas en MySQL, no solo del reloj del cliente.

- Persistir la selección aleatoria de preguntas de cada intento: recargar la página no debe cambiar las preguntas.
- Persistir el inicio de cada estación y la última estación visitada para sobrevivir recargas.

Estructura sugerida una vez creado el scaffold:

```
app/
  __init__.py          # app factory, configuración y login de admin
  db.py                # conexión MySQL y helpers
  routes/
    juego.py           # bienvenida, estaciones, transición, final, abandono
    ranking.py         # ranking público / vista de TV
    estadisticas.py    # página pública de estadísticas
    admin.py           # login y CRUD del panel de administración
  templates/
    bienvenida.html
    estacion.html
    transicion.html
    final.html
    ranking.html
    estadisticas.html
    eliminado.html
    admin/
  static/
    css/
    js/
    audio/
    vendor/            # librerías locales, sin CDN
tests/
bd/
  schema.sql           # estructura de la base (MySQL 8)
  seeds/
    001_contenido_inicial.sql  # contenido inicial idempotente y
configuración
requirements.txt
.env.example
README.md
```

Ajustar la estructura si el proyecto real adopta otra, pero mantener la separación entre rutas, plantillas, estáticos y acceso a datos.

# Modelo de datos requerido

Basado en `docs/escape-room.md`:

| Entidad | Campos mínimos |
|---|---|
| `equipo` | `id`, `nombre`, `cantidad_personas` (1–4, obligatorio) |
| `intento` | `id`, `equipo_id`, `dificultad` (`inicial`, `intermedio`, `avanzado`), `fecha_inicio`, `fecha_fin`, `se_completo_ok`, `cantidad_ayudas`, `puntaje_obtenido`, `finalizado` (`completo`, `abandono`, `por_respuestas_incorrectas`, `por_tiempo`), `ultima_estacion_visitada` |
| `estacion` | `id`, `nombre`, `descripcion`, `orden`, `activo` |
| `tema` | `id`, `nombre`, `estacion_id`, `activo` |
| `pregunta` | `id`, `titulo`, `descripcion`, `tema_id`, `dificultad` (`inicial`, `intermedio`, `avanzado`), `activo` |
| `respuesta` | `id`, `contenido`, `correcta` (booleano), `pregunta_id`, `activo` |
| `respuesta_elegida` | `id`, `fecha_respondida`, `intento_id`, `intento_envio_id`, `respuesta_id` |
| `incorrecta` | `id`, `pregunta_id`, `respuesta_id`, `fecha`, `intento_id`, `intento_envio_id` |

Criterios:

- Usar `snake_case` para tablas y columnas, y claves foráneas `<entidad>_id`.
- Guardar **todas** las respuestas elegidas en `respuesta_elegida`, correctas e incorrectas.
- Guardar además en `incorrecta` cada respuesta incorrecta, tal como pide `docs/escape-room.md`. La duplicación es intencional: `respuesta_elegida` es el registro completo y `incorrecta` el subconjunto que alimenta el HUD y las estadísticas.
- No guardar ni mostrar la respuesta correcta en el HTML/JS del participante.
- `intento.fecha_fin` queda `NULL` mientras el intento está en curso.
- `intento.se_completo_ok` es derivado: verdadero solo si `finalizado = 'completo'`. No usar dos fuentes de verdad distintas.
- **Solo los intentos con `finalizado = 'completo'` participan del ranking competitivo.** `fecha_fin` no nula **no alcanza**: abandonos y eliminaciones también tienen fecha de fin.

- `puntaje_obtenido` se completa solo para intentos completados (ver "Reglas de puntaje y ranking").
- `equipo.cantidad_personas` es obligatorio, entre 1 y 4, y se carga en la bienvenida.
- `equipo.nombre` es único (la colación `utf8mb4_unicode_ci` compara sin distinguir mayúsculas ni acentos). Si el nombre ya existe, la bienvenida pide el `equipo_id` para continuar con ese equipo; no se crea un equipo duplicado.
- `intento.finalizado` queda `NULL` mientras el intento está en curso y se completa con uno de los cuatro valores al terminar.
- `intento.se_completo_ok` se implementa como columna generada a partir de `finalizado` (no se inserta ni se edita a mano).
- `intento.dificultad` guarda el nivel elegido y no cambia durante el intento.
- `estacion.orden` define el recorrido 1 a 5 de forma explícita; no depender del `id`.
- `estacion`, `tema`, `pregunta` y `respuesta` usan **borrado lógico** con `activo`. El juego y los selectores filtran `activo = 1`; los intentos históricos siguen leyendo las filas desactivadas.
- `respuesta_elegida.intento_envio_id` e `incorrecta.intento_envio_id` son una extensión al modelo base para agrupar las respuestas de un mismo envío del formulario.
- `equipo`, `intento`, `respuesta_elegida` e `incorrecta` son históricos: no se borran, ni física ni lógicamente.
- Respetar el **máximo de 4 temas por estación**.
- La clave primaria `id` debe ser auto incremental.

# Tablas de detalle necesarias

El modelo de `docs/escape-room.md` no alcanza para cronometrar cada estación, contar envíos, persistir la selección aleatoria, habilitar dificultades ni registrar ayudas. Agregar estas tablas **sin reemplazar** el modelo base:

```
intento_estacion(id, intento_id, estacion_id, fecha_inicio, fecha_fin,
ayudas, completada)
intento_envio(id, intento_id, estacion_id, fecha, respuestas_correctas,
respuestas_incorrectas)
intento_pregunta(id, intento_id, estacion_id, pregunta_id, orden)   #
selección aleatoria estable
configuracion(clave, valor)                                          #
dificultades habilitadas y otros parámetros
```

- `intento_envio` es imprescindible para mostrar "cantidad de intentos de envío del formulario"; no intentar derivarlo solo de `respuesta_elegida`.
- `intento_pregunta` garantiza que al recargar se muestren las mismas 4 preguntas y que el resultado sea reproducible.
- `configuracion` guarda, por ejemplo, `dificultad_intermedia_habilitada` y `dificultad_avanzada_habilitada`, además de los parámetros del juego (`duracion_estacion_segundos`, `max_ayudas_estacion`, `max_envios_estacion`, `max_respuestas_incorrectas`, `max_integrantes_equipo`, `max_temas_por_estacion`, `preguntas_por_estacion`). La dificultad `inicial` queda siempre habilitada.
- Un nivel solo puede habilitarse si todas las estaciones tienen al menos 1 pregunta activa de ese nivel por tema y 4 en total.
- **No existe tabla `pista`.** Las pistas están en `docs/ESCAPE ROOM.pdf`, las leen los estudiantes, y la app solo registra 1 o 2 ayudas por estación en `intento_estacion.ayudas` y en `intento.cantidad_ayudas`.
- Cualquier tabla extra debe documentarse en `README.md` y en este archivo.

# Flujo de pantallas

1. **Bienvenida:** formulario con nombre de equipo, cantidad de integrantes (1–4) y dificultad. Validar nombre no vacío y cantidad entre 1 y 4. Solo se ofrecen las dificultades habilitadas en `configuracion`. Si el nombre ya existe, pedir el **ID del equipo** (el que se muestra siempre en el HUD) y continuar con ese equipo; si no existe, crear `equipo` e `intento`. Al reingresar con un equipo que tiene un intento activo (`fecha_fin IS NULL`), reanudar ese intento; si no, crear uno nuevo. Mostrar el ID del equipo en todas las pantallas del intento.
2. **Estación 1 a 5:** una página/ruta por estación, con sus 4 preguntas, el temporizador y el HUD (ayudas, incorrectas, envíos). No permitir saltar estaciones por URL.
3. **Transición:** confirmar estación superada y permitir continuar.
4. **Final:** mostrar resultado del intento y el ranking según las reglas de abajo.
5. **Ranking público:** vista de solo lectura, pensada para el televisor. Debe actualizarse sola (por ejemplo, con polling) y ser legible a distancia.
6. **Eliminación:** pantalla clara cuando el equipo queda fuera por tiempo, respuestas incorrectas o abandono.

7. **Estadísticas públicas:** solo agregados (participantes, integrantes, intentos, tiempo promedio y respuestas realizadas / correctas). Sin nombres de equipos, sin detalle por pregunta y sin respuestas correctas. Accesible sin login.
8. **Login de administración:** `/admin`, con usuario y clave.
9. **Panel de administración:** cards, gráficos y CRUD de las tablas.

# Administración

La administración es un **requisito**, no un extra opcional.

- Ruta `/admin`, **no enlazada** desde ninguna página pública.
- Login con usuario y clave; sin login no se accede a ninguna ruta `/admin/*`.
- Credenciales por variables de entorno: `ADMIN_USER` y `ADMIN_PASSWORD_HASH` (usar `werkzeug.security.generate_password_hash`). Nunca hardcodear claves ni commitearlas.
- Sesión de Flask para mantener el login; incluir cierre de sesión.
- Panel de control con cards de Bootstrap, una por tabla, y acceso a los listados.
- Gráficos estadísticos: equipos participantes, intentos finalizados, tiempo promedio por estación, tiempo promedio de los intentos finalizados, porcentaje de utilización de cada pregunta sobre el total de intentos (completados o no) y porcentaje de aciertos.
- CRUD de todas las tablas del modelo base.
- En los listados, permitir filtros y ordenamiento por `id` y por otros campos relevantes (nombre, dificultad, estación, tema, fechas).
- Habilitar/deshabilitar las dificultades `intermedio` y `avanzado` desde el panel, persistiendo en `configuracion`.
- **Solo se puede deshabilitar una dificultad si no hay equipos jugando** (`intento.fecha_fin IS NULL`). El panel muestra los intentos activos y bloquea el toggle hasta que terminen.
- Un nivel solo se puede habilitar si **todas** las estaciones tienen cobertura completa de ese nivel: al menos 1 pregunta por tema activo y al menos 4 en total.
- Mostrar un reporte de cobertura estación × nivel con los conteos y marcar en rojo lo que falta.
- Validar el **máximo de 4 temas por estación**.
- Borrado **lógico** (`activo = 0`) en `estacion`, `tema`, `pregunta` y `respuesta`; los listados permiten filtrar activos e inactivos.
- Bloquear el borrado de `equipo`, `intento`, `respuesta_elegida` e `incorrecta`: son históricos.

- Cambiar la respuesta correcta de una pregunta no debe alterar intentos ya finalizados.

# Reglas de puntaje y ranking

Según `docs/escape-room.md`, el ranking se ordena por:

1. **Menor tiempo total** para completar las 5 estaciones.
2. A igual tiempo, **menor cantidad de ayudas**.
3. Desempate final estable por `id`.

Query de referencia (solo intentos completados):

```
WHERE finalizado = 'completo'
ORDER BY TIMESTAMPDIFF(SECOND, fecha_inicio, fecha_fin) ASC,
         cantidad_ayudas ASC,
         id ASC
```

- `puntaje_obtenido` guarda los **segundos totales** del intento completado (`TIMESTAMPDIFF(SECOND, fecha_inicio, fecha_fin)`). Para intentos no completados queda `NULL`.
- El ranking se ordena por tiempo y ayudas, **no** por `puntaje_obtenido`.
- Al mostrar un equipo que repitió intentos, indicar al lado el número de intento.
- No usar el orden del prototipo `ranking-puntajes.html` (puntos descendente y luego tiempo). Ese prototipo era una demostración con carga manual de puntos.

# Temporizadores

- El contador general se calcula como `ahora - intento.fecha_inicio` y solo aumenta.
- El contador de estación parte de 4:00 y baja hasta 0:00.
- Al llegar a 30 segundos restantes: audio + texto titilante visible.
- Al llegar a 0: el intento pasa a `finalizado = 'por_tiempo'` y no puede continuar.
- El tiempo debe sobrevivir a recargas de página. Persistir `intento_estacion.fecha_inicio` y recalcular desde el servidor.
- Al reanudar, retomar desde `intento.ultima_estacion_visitada`.

- El audio puede estar bloqueado por el navegador hasta que haya una interacción del usuario. Desbloquearlo al presionar "Iniciar" y mantener siempre la advertencia visual.
- Usar archivos locales de Bootstrap y audio: la feria puede no tener Internet confiable.

# Decisiones registradas

Estas decisiones interpretan puntos que el documento no cierra del todo. Si el equipo las redefine, actualizar esta sección y el código.

1. **Eliminación por respuestas incorrectas:** se acumulan las respuestas incorrectas del intento dentro de la estación; al llegar a **2** el intento pasa a `finalizado = 'por_respuestas_incorrectas'`. Además hay un máximo de **2 envíos** del formulario por estación: si el segundo envío no tiene todo correcto, solo queda el botón de finalizar. Un mismo envío con 2 o más incorrectas elimina de inmediato.
2. **Ranking:** participa únicamente `finalizado = 'completo'`.
3. **Puntaje:** `puntaje_obtenido` = segundos totales del intento completado; `NULL` en el resto.
4. **Selección de preguntas:** siempre 4, del nivel del intento. Se elige al menos 1 por tema y se completa al azar; con más de 4 temas se sortean 4 temas y 1 pregunta de cada uno. Se persiste en `intento_pregunta`.
5. **Dificultades:** `inicial` siempre habilitada. `intermedio` y `avanzado` solo se habilitan si todas las estaciones tienen cobertura de ese nivel y solo se deshabilitan si no hay equipos jugando.
6. **Admin:** un único usuario configurado por variables de entorno.
7. **Gráficos:** Chart.js incluido localmente en `static/vendor/` (sin CDN). Si se prefiere cero dependencias, reemplazar por barras SVG/CSS generadas en el servidor.
8. **`se_completo_ok`:** derivado de `finalizado`; no se edita a mano.
9. **Cantidad de integrantes:** se pide en la bienvenida, es obligatoria y va de 1 a 4.
10. **Borrado:** lógico (`activo`) en el contenido; bloqueado en los históricos.
11. **Estadísticas públicas:** agregados sin nombres de equipos, sin detalle por pregunta y sin respuestas correctas.
12. **Dificultad del intento:** queda guardada en `intento.dificultad` y no cambia durante el intento.
13. **Máximo de temas:** 4 por estación.

14. **Regla de contenido:** se usa "al menos 1 pregunta por tema activo y 4 por estación", no la lectura literal de "mínimo 4 preguntas por tema" de `docs/escape-room.md`. El contenido inicial (14 temas, 20 preguntas) cumple esta regla.
15. **Ayudas:** la app **no muestra pistas**. Por estación y por intento se pueden registrar hasta 2 ayudas con los botones **Ayuda 1** y **Ayuda 2**; cada una suma en `intento_estacion.ayudas` y en `intento.cantidad_ayudas`. Al llegar a 2 se deshabilitan. Las pistas las leen los estudiantes desde `docs/ESCAPE ROOM.pdf`.
16. **Identidad del equipo:** `equipo.nombre` es único. Si el nombre ya existe, la bienvenida pide el ID del equipo y no crea uno nuevo. El ID se muestra siempre. Cada reingreso crea un intento nuevo, salvo que el equipo tenga un intento activo (`fecha_fin IS NULL`), en cuyo caso se reanuda. Si dos equipos distintos quieren el mismo nombre, deben usar un sufijo.

# Convenciones de código

- UI y textos en **español rioplatense**, consistentes con el prototipo ("Ingresá", "Respondé", "Superá").
- Nombres de dominio en español (`equipo`, `intento`, `estacion`, `pregunta`, `respuesta`); nombres técnicos pueden ir en inglés.
- Rutas Flask con blueprints y app factory.
- SQL siempre parametrizado. Nunca interpolar datos del usuario.
- Transacciones para registrar un intento, envíos, respuestas y ayudas.
- Configuración por variables de entorno; incluir `.env.example` y nunca commitear credenciales.
- Bootstrap y JS local en `static/`; no depender de CDN para el funcionamiento en el aula.
- Preguntas, opciones y respuestas correctas deben vivir en la base de datos y cargarse con un seed idempotente (`bd/seeds/001_contenido_inicial.sql`). No duplicarlas en plantillas ni en varios archivos JS. Las pistas no se guardan en la base: están en el PDF y las leen los estudiantes.
- Las preguntas del prototipo y de `docs/ESCAPE ROOM.pdf` son la fuente de contenido; unificar diferencias antes de cargarlas.
- No agregar autenticación de participantes ni registro público. El único login es el de administración.

- La regla de **máximo 3 equipos simultáneos** y el ingreso de un nuevo equipo cuando otro llega a la estación 3 es operativa. No imponerla en el software sin confirmación.
- La página de estadísticas pública es de solo lectura; no expone respuestas correctas ni datos personales.
- Código simple, nombres descriptivos y comentarios solo donde aporten.
- Diseño limpio y responsive; cualquier pantalla nueva debe verse bien en el móvil.

# Comandos previstos

El esquema y el seed ya existen y se pueden ejecutar. El resto de los comandos se completa cuando se cree el scaffold; documentar los reales en `README.md`.

```
mysql -u <usuario> -p < bd/schema.sql
mysql -u <usuario> -p < bd/seeds/001_contenido_inicial.sql

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
flask --app app run --debug
pytest
```

No inventar comandos de arranque ni dependencias que no estén implementadas.

# Pruebas y verificación

Priorizar pruebas automatizadas para:

- Avance solo con las 4 respuestas correctas.
- Eliminación por 2 respuestas incorrectas.
- Eliminación por tiempo agotado.
- Eliminación por abandono.
- Orden del ranking: tiempo ascendente y, en empate, menos ayudas.
- Exclusión de intentos no completados (`abandono`, `por_respuestas_incorrectas`, `por_tiempo`).
- Filtrado de preguntas por dificultad y por tema.
- Bienvenida con cantidad de integrantes entre 1 y 4.
- Reingreso con un nombre de equipo existente: pide el ID y no duplica el equipo.

- Ayudas: máximo 2 por estación e intento, con contador visible y botones deshabilitados al llegar a 2.
- La app no muestra ni envía el texto de las pistas.
- Selección de 4 preguntas con al menos una por tema y 4 en total.
- Con más de 4 temas, se usan 4 temas al azar y una pregunta de cada uno.
- Nivel no habilitable si falta cobertura en alguna estación o tema.
- Toggle de dificultad bloqueado mientras hay intentos activos.
- Borrado lógico de contenido sin romper intentos históricos.
- Estadísticas públicas sin nombres de equipos ni respuestas correctas.
- Selección aleatoria estable tras recargar (mismas 4 preguntas del intento).
- Registro de todas las respuestas elegidas y de las incorrectas en `incorrecta`.
- Conteo de envíos del formulario y bloqueo al segundo envío.
- Recarga de página sin reiniciar tiempos.
- Reanudación desde `ultima_estacion_visitada`.
- Ausencia de respuestas correctas en el HTML/JS enviado al cliente.
- `/admin` inaccesible sin login y no enlazado desde páginas públicas.
- Habilitar/deshabilitar dificultades, su efecto en la bienvenida y el bloqueo con intentos activos.
- Cálculo de estadísticas y gráficos.
- Seed idempotente: ejecutarlo dos veces no duplica datos.

Verificación manual antes de entregar:

- Bienvenida con nombre, cantidad de integrantes (1–4) y dificultad, mostrando solo dificultades habilitadas.
- Las 5 estaciones se recorren en orden.
- HUD con ayudas, incorrectas y envíos.
- Contador general sube y contador de estación baja.
- Aviso visual y sonoro a los 30 segundos.
- Pantalla de eliminación y pantalla final.
- Ranking visible y actualizado en la vista de TV.
- Página pública de estadísticas.
- Panel `/admin` con login, CRUD, filtros, orden y gráficos.
- Funcionamiento sin conexión a Internet.

# Qué no hacer

- No modificar ni eliminar `docs/`, sus PDF ni los prototipos.

- No presentar `docs/src/index.html` como la aplicación final ni copiar su lógica de respuestas en el cliente.
- No reutilizar `basededatos.py`, `basededatos_contador.db` ni `contador.php`; el stack es Flask + MySQL.
- No ordenar el ranking por puntos como en el prototipo de ranking.
- No enlazar `/admin` desde la interfaz pública ni dejar sus rutas sin autenticación.
- No aleatorizar preguntas sin persistir la selección por intento.
- No borrar físicamente contenido ni filas históricas desde el CRUD; usar borrado lógico o bloquear.
- No habilitar un nivel sin cobertura completa en todas las estaciones.
- No deshabilitar una dificultad mientras haya equipos jugando.
- No mostrar nombres de equipos, detalle por pregunta ni respuestas correctas en la página pública de estadísticas.
- No introducir React/Vue, ORM pesado, Docker u otra infraestructura sin necesidad y sin consultar.
- No usar CDN para Bootstrap, gráficos ni audio.
- No hardcodear preguntas, respuestas, credenciales ni tiempos en múltiples lugares.
- No commitear `.env`, volcados de base de datos ni archivos generados.
- No mostrar ni cargar pistas en la aplicación: solo se registran las ayudas.
- No permitir dos equipos con el mismo nombre ni crear un equipo nuevo cuando la bienvenida pide el ID.

# Forma de trabajar

- Haz solo lo que se pide: no añadas funcionalidades por tu cuenta.
- Cambios pequeños y enfocados; no reescribas lo que ya funciona.
- Al terminar, resume qué has cambiado y cualquier decisión que deba revisar.

# Límites

- Siempre: respetar las reglas y mantener los textos en español.
- Pregunta antes: crear archivos nuevos, cambiar el formato de los datos guardados.
- Nunca: añadir dependencias, frameworks o un paso de build que no haya establecido.